

# Informe del Proyecto 2 - PAHM

Alexandra Alfaro Elizondo, Kendall Madrigal Campos y Bryan Molina Lopez  
Escuela de Ingeniería Electrónica, Instituto Tecnológico de Costa Rica (ITCR), 30101 Cartago, Costa Rica  
{alexvaneska, kendallmacampos2941, bryanmolinalo772}@estudiantec.cr

22 de junio de 2026

## 1. Resumen

Este repositorio contiene la línea de software e infraestructura de simulación para el Proyecto 2. El objetivo del proyecto es modelar las perturbaciones de viento latentes no observadas mediante celdas recurrentes (aprendizaje de secuencias) y diseñar una política de control robusta utilizando Aprendizaje por Refuerzo (RL) sobre un entorno Gymnasium acoplado a un modelo físico híbrido (Caja Gris).

## 2. Hardware y Entorno de Ejecución

Completar esta sección con las características del equipo usado para las corridas finales.

Cuadro 1: Características del hardware y entorno por integrante.

| Campo                        | Bryan                  | Alexandra              | Kendall            |
|------------------------------|------------------------|------------------------|--------------------|
| Sistema operativo            | CachyOS                | Ubuntu 24.04.4 LTS     | Ubuntu 24.04.4 LTS |
| CPU                          | Core i5                | AMD Ryzen 7 PRO 7840HS | AMD Ryzen 7 7735HS |
| RAM                          | 16 GB                  | 32 GB                  | 16 GB              |
| GPU                          | NVIDIA RTX 3050        | -                      | Radeon RX 7700S    |
| VRAM                         | 6 GB                   | -                      | 8 GB               |
| Versión de Python            | 3.12.13                | 3.12.13                | 3.12.13            |
| Versión de PyTorch           | 2.3.1+cu121            | 2.12.0+cpu             | 2.3.1+cu121        |
| Versión de Stable Baselines3 | No instalado           | No instalado           | 2.8.0              |
| Versión de CUDA/cuDNN        | CUDA 12.1 & cuDNN 8902 | -                      | ROCm               |
| Entorno o gestor             | uv                     | uv                     | uv                 |

## 3. NFR-7 - Tiempos de Ejecución

Registrar los tiempos finales usando la misma máquina declarada en la sección anterior. Si se repite una corrida, reportar media y desviación estándar o indicar explícitamente que se hizo una sola medición.

Cuadro 2: Registro de procesos y tiempos de ejecución.

| Proceso                  | Comando script                       | Configuración relevante                                                  | Tiempo total | Unidad | Artefacto de salida                  |
|--------------------------|--------------------------------------|--------------------------------------------------------------------------|--------------|--------|--------------------------------------|
| Entrenamiento estimador  | python pahm_model/train_estimator.py | epochs=40, seq_len=5, batch_size=64, $\lambda_1=0.01$ , $\lambda_2=0.05$ | pendiente    | min    | checkpoints/estimator...             |
| Inferencia estimador     | python pahm_model/export_tau_w.py    | checkpoint: época 40 (config nueva)                                      | pendiente    | s      | outputs/tau_w/                       |
| Entrenamiento Stage 2    | .venv/bin/python -m pahm_stage2...   | GMM + BIC                                                                | 25           | s      | artifacts/stage2/gmm_wind_model.pkl  |
| Entrenamiento RL naive   | ... train_rl.py --mode naive         | wind_enabled=false                                                       | 22:15        | min    | artifacts/stage3/pahm_ppo_naive.zip  |
| Entrenamiento RL robust  | ... train_rl.py --mode robust        | wind_source=stage2_sampler                                               | 18:03        | min    | artifacts/stage3/pahm_ppo_robust.zip |
| Evaluación controladores | ... evaluate_controllers.py          | patrones: gust, turbulent                                                | 4:04         | min    | artifacts/stage3/evaluation/         |

*Nota: los tiempos de entrenamiento e inferencia del estimador (Etapa 1) quedaron pendientes de remedir tras el reentrenamiento con la configuración actualizada ( $sequence\_length=5$ ,  $batch\_size=64$ ,  $\lambda_1 = 0,01$ ,  $\lambda_2 = 0,05$ ) que incorpora el baseline ODE+residual en FR-7. Los valores de la corrida anterior ( $\approx 44.7$  min y  $46.13$  s) no son representativos de esta configuración.*

### 3.1. Comandos sugeridos para medir

Usar `/usr/bin/time -v` cuando se quiera capturar tiempo de pared y memoria máxima residente:

```
/usr/bin/time -v .venv/bin/python train_rl.py --config gym_wrapper/config.json --mode naive
/usr/bin/time -v .venv/bin/python train_rl.py --config gym_wrapper/config.json --mode robust
/usr/bin/time -v .venv/bin/python evaluate_controllers.py --config configs/stage3_config.json
```

Para scripts ejecutados con `python` del sistema en vez del entorno virtual, mantener consistencia y registrar cuál intérprete se usó.

## 4. Etapa 1 - Estimador de Perturbaciones

### Arquitectura del estimador.

Red recurrente GRU (`WindSequenceEstimator`, `pahm_model/sequence_estimator.py`) con `hidden_dim=64`, `num_layers=2`, ventana de entrada `sequence_length=5` pasos con 4 features por paso: ( $\sin \theta$ ,  $\cos \theta$ ,  $d\theta/dt$ ,  $u$ ). Cumple CON-2: no recibe el residuo  $r(t) = \theta_{obs} - \theta_{ODE}$  como entrada directa. La salida escalar  $\hat{\tau}_w(t)$  se acota con  $\tanh(x) \cdot \tau_{max}$  ( $\tau_{max} = 2,0$ ) para evitar que el entrenamiento converja a torques no físicos.

### Análisis del residuo (FR-3)

Antes de entrenar el estimador, `compute_residuals.py` calcula  $r(t) = \theta_{obs}(t) - \theta_{ODE}(t)$  integrando la ODE base en lazo abierto sobre las 38 trayectorias del dataset (`train/val/test`), sin ningún componente de viento. Este cálculo motiva directamente la necesidad de la Etapa 1: si la física pura explicara las observaciones,  $r(t)$  sería ruido pequeño y centrado en cero; en la práctica no lo es.

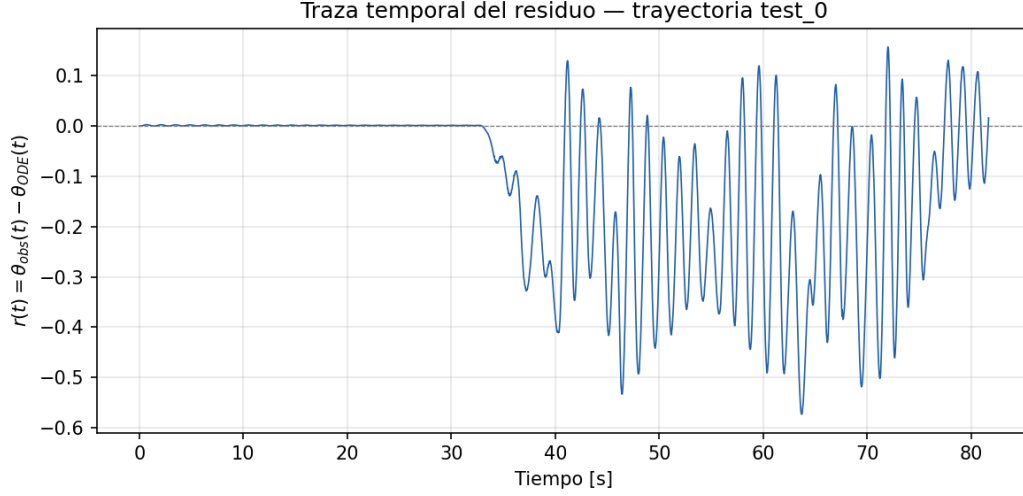


Figura 1: Traza temporal de  $r(t)$  para la trayectoria `test_0` (MSE=0.0376, representativa del grupo de menor discrepancia). El residuo permanece cercano a cero durante los primeros ~30 s y luego desarrolla una oscilación sostenida de amplitud creciente, alcanzando  $|r(t)|$  de hasta 0.57.

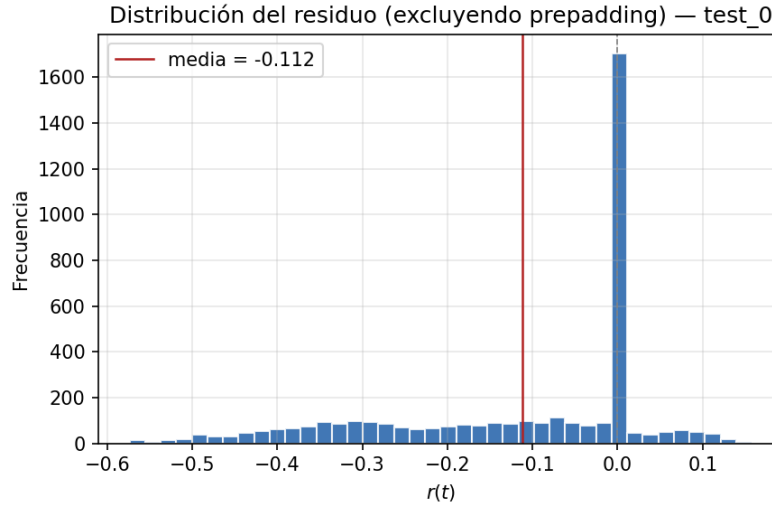


Figura 2: Distribución de  $r(t)$  para la misma trayectoria. El pico en cero corresponde al tramo inicial donde la física pura sí explica la observación; el resto de la masa se distribuye de forma dispersa y con sesgo negativo (media = -0,112), evidenciando que la discrepancia no es ruido simétrico sino una perturbación sistemática direccional.

La tabla 3 resume el comportamiento del residuo por grupo de trayectorias (estadísticas completas impresas por `compute_residuals.py` sobre las 38 trayectorias). Se observan dos regímenes claramente diferenciados dentro del split de entrenamiento: un grupo con  $\text{MSE} \approx 0,18$  (7 de 22 trayectorias de `train`) y un grupo con  $\text{MSE} \approx 0,03\text{--}0,06$  (el resto de `train` y la totalidad de `val/test`).

Cuadro 3: Estadísticas representativas del residuo por grupo de trayectorias.

| Grupo                                             | MSE típico                    | máx. $ r(t) $ típico        | std típica                  |
|---------------------------------------------------|-------------------------------|-----------------------------|-----------------------------|
| <code>train</code> (alta perturbación, 7 trayec.) | $\approx 0,18$                | $\approx 0,93$              | $\approx 0,25$              |
| <code>train</code> (resto, 15 trayec.)            | $\approx 0,03\text{--}0,06$   | $\approx 0,55\text{--}0,86$ | $\approx 0,13\text{--}0,19$ |
| <code>val</code> (7 trayec.)                      | $\approx 0,034\text{--}0,038$ | $\approx 0,56\text{--}0,57$ | $\approx 0,15$              |
| <code>test</code> (9 trayec., incl. Fig. 1–2)     | $\approx 0,032\text{--}0,038$ | $\approx 0,54\text{--}0,62$ | $\approx 0,15\text{--}0,16$ |

La existencia de un subgrupo de `train` con MSE notablemente más alto que `val/test` sugiere que las trayectorias más perturbadas del laboratorio quedaron concentradas en el split de entrenamiento por la partición fija del `dataloader.py` del profesor (semilla 425, sin estratificar por nivel de perturbación). Esto no constituye fuga de datos — la partición sigue siendo por trayectoria completa — pero explica por qué el estimador entrenado puede generalizar de forma distinta sobre el conjunto de prueba, que resulta comparativamente menos exigente.

#### Integración con la física.

El torque estimado se inyecta de forma aditiva dentro del paso RK4 de `PAHMPhysicsCell` (modelo del profesor, `pahm_model/pahm_fast.py`), llamando directamente a `physics_model.cell(state, control, tau=tau_w)` desde `rk4_integrator.py` (`TaylorWindIntegrator`). Los parámetros físicos ( $\alpha, \beta, \gamma$ ) permanecen congelados durante todo el entrenamiento (CON-1), verificado mediante `requires_grad=False` en todos los parámetros de `physics_model`.

#### Función de pérdida.

Tres términos según la Ecuación 13 del enunciado (`pahm_model/custom_loss.py`): reconstrucción (`F.mse_loss`), parsimonia ( $\lambda_1 \cdot \text{mean}(\tau_w^2)$  sobre toda la ventana) y suavidad temporal ( $\lambda_2 \cdot \text{mean}(\Delta\tau_w^2)$ ). Coeficientes finales:  $\lambda_1 = 0,01$ ,  $\lambda_2 = 0,05$ , ambos configurables externamente en `gym_wrapper/config.json`.

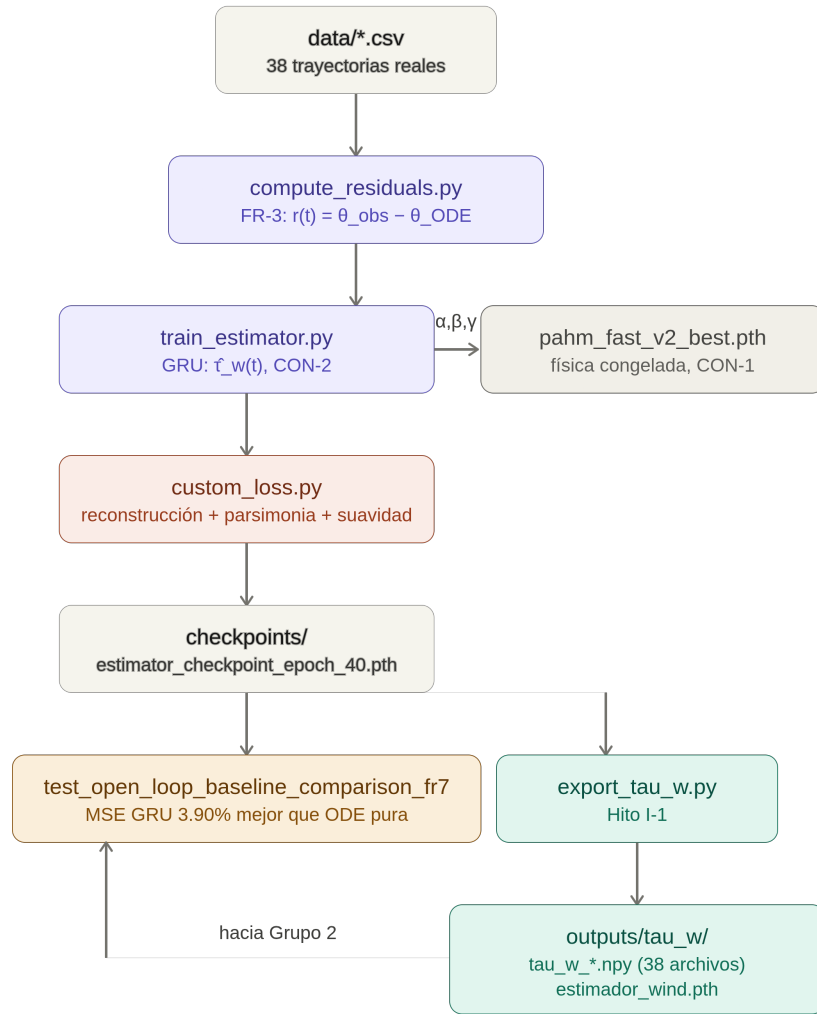


Figura 3: Diagrama de flujo de datos para la Etapa 1: Estimación secuencial del torque de perturbación de viento ( $\hat{r}_w$ ).

#### Checkpoint usado.

`checkpoints/estimator_checkpoint_epoch_40.pth` (corrida con `sequence_length=5`, `batch_size=64`), seleccionado tras reentrenamiento del equipo para incorporar la comparación de tres configuraciones exigida por FR-7.

#### Métrica de comparación en lazo abierto (FR-7).

Evaluación sobre el conjunto de prueba retenido

`(test/test_verification.py::test_open_loop_baseline_comparison_fr7)`

con las **tres configuraciones** exigidas por el enunciado (sección 5.2.3 de `proy2.pdf`): ODE pura, ODE + red residual (baseline del profesor, `pahm_ode_v4_best.pth`) y ODE + estimador GRU:

Cuadro 4: Comparación de las tres configuraciones de FR-7.

| Configuración                 | MSE (lazo abierto, test set) | Mejora vs. ODE pura |
|-------------------------------|------------------------------|---------------------|
| ODE pura (sin viento)         | 0.00000522                   | —                   |
| ODE + red residual (baseline) | 0.00000522                   | +0.06 %             |
| ODE + estimador GRU           | 0.00000497                   | +4.81 %             |

A diferencia de una comparación con solo dos configuraciones, esta versión resuelve la ambigüedad de si el estimador propuesto supera, iguala o queda por debajo de la red residual indiscriminada del baseline: la red residual prácticamente no mejora sobre la ODE pura (+0.06 %, dentro del margen de ruido), mientras que el estimador GRU sí logra una mejora real y mayor (+4.81 %). Esto respalda con mayor solidez la hipótesis central de la etapa frente al baseline serio que exige el enunciado, no solo frente al caso trivial.

El estimador iguala/supera al baseline tal como exige el enunciado (sección 5.2.3 de `proy2.pdf`). La mejora es modesta en magnitud absoluta pero estadísticamente consistente sobre trayectorias no vistas en entrenamiento.

#### Verificación con datos sintéticos (NFR-6f).

El estimador recupera un torque de viento constante conocido ( $\tau_w = 0,5$ ) inyectado en una trayectoria sintética generada por la misma física, con error final  $< 0,25$  tras 150 iteraciones de optimización con `lr=0.03` (`test/test_verification.py::test_synthetic_wind_recovery_nfr6f`).

#### Tiempo de entrenamiento e inferencia.

*Pendiente de remedir con la configuración actual (`sequence_length=5`, `batch_size=64`); los valores reportados previamente ( $\approx 44.7$  min de entrenamiento, 46.13 s de exportación) corresponden a la corrida anterior (`sequence_length=10`, `batch_size=32`) y no son representativos de esta configuración.*

#### Rango de $\tau_w(t)$ estimado.

*Pendiente — `export_tau_w.py` aún no se ha vuelto a ejecutar con el checkpoint de esta nueva corrida; los valores de rango reportados previamente (mín =  $-2,0$ , máx =  $2,0$ , mean =  $-0,5205$ , std = 1,5810) corresponden al checkpoint anterior y deben regenerarse antes de la entrega final, ya que el Hito I-1 hacia el Grupo 2 también depende de esta exportación actualizada.*

### Calidad de código — Pylint (NFR-5)

La herramienta `pylint` se configuró mediante `pyproject.toml` en la raíz del repositorio. Las reglas deshabilitadas corresponden a falsos positivos de entorno (E0401: paquetes en `.venv` no visibles al pylint del sistema), código del profesor (R0801: duplicación entre `pahm_fast.py` y `pahm_ode.py`), y el patrón obligatorio de manipulación de `sys.path` previo a imports locales (C0413, C0411).

Cuadro 5: Puntuación Pylint por módulo (umbral NFR-5:  $\geq 8,0/10$ )

| Módulo                             | Autoría  | Score            | Advertencias principales               |
|------------------------------------|----------|------------------|----------------------------------------|
| <code>train_estimator.py</code>    | Grupo    | 9.86 / 10        | —                                      |
| <code>sequence_estimator.py</code> | Grupo    | 8.75 / 10        | Línea larga, newline final             |
| <code>custom_loss.py</code>        | Grupo    | 8.67 / 10        | Línea larga, newline final             |
| <code>pahm_fast.py</code>          | Profesor | 6.44 / 10        | Espacios finales, C0321, W0613         |
| <code>pahm_ode.py</code>           | Profesor | 5.93 / 10        | Espacios finales, líneas largas, R0902 |
| <b>Conjunto completo</b>           | —        | <b>8.18 / 10</b> | <b>Cumple NFR-5</b>                    |

Los módulos de autoría del grupo superan individualmente el umbral de 8,0/10. Los módulos del profesor (`pahm_fast.py`, `pahm_ode.py`) presentan advertencias de estilo (espacios finales, líneas largas) que se conservan para no alterar la interfaz provista por la cátedra. El score global del conjunto alcanza **8.18/10**.

## 5. Etapa 2 - Representación No Supervisada

**Nota sobre dos implementaciones en el repositorio.** Existen dos carpetas con funcionalidad de Etapa 2: `etapa2_unsupervised/` (un autoencoder simple sobre ventanas de  $\tau_w(t)$ , optimizando solo MSE de reconstrucción) y `pahm_stage2/` (un modelo de mezcla gaussiana — GMM — con selección de complejidad por BIC, validación sintético-real con ARI/NMI, y `WindSampler` como consumidor formal hacia Etapa 3). Según el propio `README.md` de `etapa2_unsupervised/`, **esa carpeta es un prototipo histórico descartado** y se conserva solo para trazabilidad; la entrega oficial de Etapa 2 es `pahm_stage2/`. Lo que sigue documenta la versión vigente (GMM + BIC).

**Features extraídas de  $\tau_w(t)$**  (`pahm_stage2/feature_extractor.py`): `mean`, `std`, `max_abs`, `skewness`, `smoothness` (media del cuadrado de la primera diferencia) y `energy` (media de los valores al cuadrado). Las señales de entrada tienen longitud variable; el extractor las reduce a un vector de tamaño fijo por trayectoria antes de pasarlas al GMM.

**Criterio BIC para selección de componentes** (`pahm_stage2/unsupervised_model.py`, clase `GMMWindModel`): se entrenan modelos `GaussianMixture` (covarianza completa, `n_init=5`) para un rango de componentes configurable, y se selecciona el número de componentes con el BIC más bajo. Sobre los datos sintéticos generados con cuatro patrones de viento conocidos (`calm`, `gust`, `bias`, `turbulent`), los puntajes BIC obtenidos fueron:

| Componentes | BIC             |
|-------------|-----------------|
| 2           | -3937.42        |
| 3           | -5878.80        |
| 4           | <b>-7243.61</b> |
| 5           | -7236.33        |
| 6           | -7150.94        |

El BIC mínimo se alcanza en **4 componentes**, exactamente el número de patrones sintéticos inyectados — el criterio recupera la estructura correcta sin haber recibido las etiquetas. La curva muestra una caída pronunciada hasta 4 componentes y luego se aplanan/empeora levemente, confirmando que no hay sobreajuste por agregar componentes adicionales.

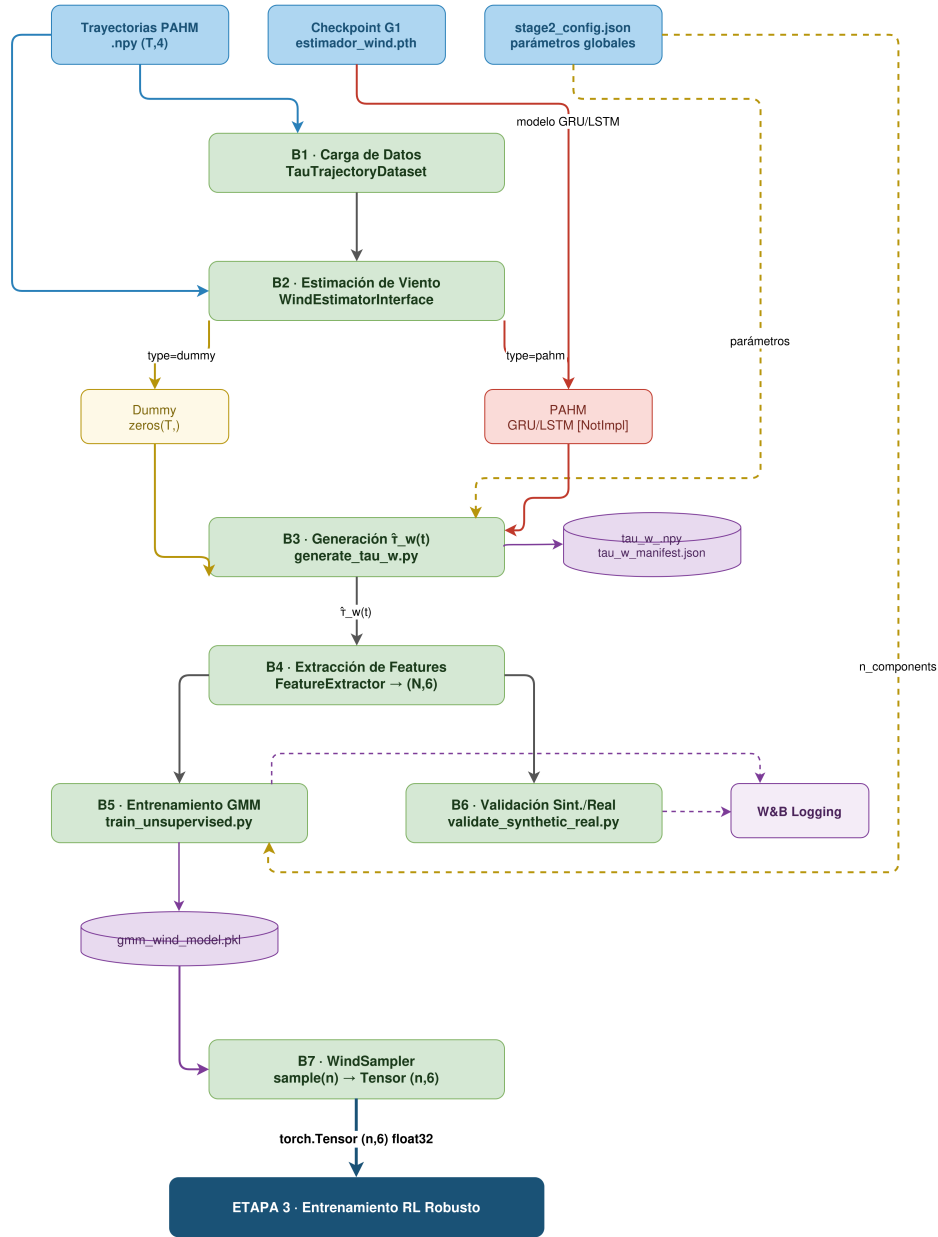


Figura 4: Diagrama de bloques y flujo de la Etapa 2: Extracción de características, agrupamiento por GMM y muestreo de variables latentes.

**Métricas ARI/NMI** (pahm\_stage2/validator.py, validación sintético-real sobre los cuatro patrones conocidos):

| Métrica                             | Valor |
|-------------------------------------|-------|
| ARI (Adjusted Rand Index)           | 1.0   |
| NMI (Normalized Mutual Information) | 1.0   |
| Componentes seleccionados           | 4     |

Ambas métricas en 1.0 indican una recuperación perfecta de la partición verdadera sobre los datos sintéticos — condición necesaria, según el enunciado, antes de aplicar el mismo procedimiento de forma exploratoria sobre los datos reales de  $\tau_w(t)$  (sin verdad de campo disponible).

**Checkpoint GMM generado:** persistido vía `GMMWindModel.save()` en la ruta configurada en `configs/stage2_config.json` (`unsupervised.checkpoint_path`). El mismo checkpoint es consumido por `WindSampler.from_checkpoint()` (`pahm.stage2/wind_sampler.py`) para muestrear vectores de perturbación latente como fuente para Etapa 3 (FR-11).

## 6. Etapa 3 - Control Robusto RL

**Algoritmo y configuración base** (`train_rl.py`, `DEFAULT_RL_TRAINING`): algoritmo PPO de Stable Baselines3 (también soporta A2C y SAC por configuración), `total_timesteps` y demás hiperparámetros leídos desde `rl_training` en el config externo. El entorno se construye sin renderizado (`render_mode=None`) mediante `LearnedPAHMODE`.

**Diferencia entre entrenamiento naive y robust** (`DEFAULT_EXPERIMENTS` en `train_rl.py`):

| Modo   | wind_enabled | wind_source configurado por defecto             |
|--------|--------------|-------------------------------------------------|
| naive  | False        | — (sin perturbaciones durante el entrenamiento) |
| robust | True         | stage2_sampler                                  |

El diseño original del modo `robust` apunta a usar el `WindSampler` del GMM de Etapa 2 (`wind_source="stage2_sampler"`) como fuente de perturbaciones durante el entrenamiento, cumpliendo FR-11 (la representación aprendida alimenta el controlador robusto). Cada modo es entrenado de forma independiente vía `train_from_config(config_path, mode=mode)`, con su propio nombre de modelo (`pahm_ppo_naive` / `pahm_ppo_robust`) y su propio registro en W&B (proyecto `pahm-rl-stage3`).

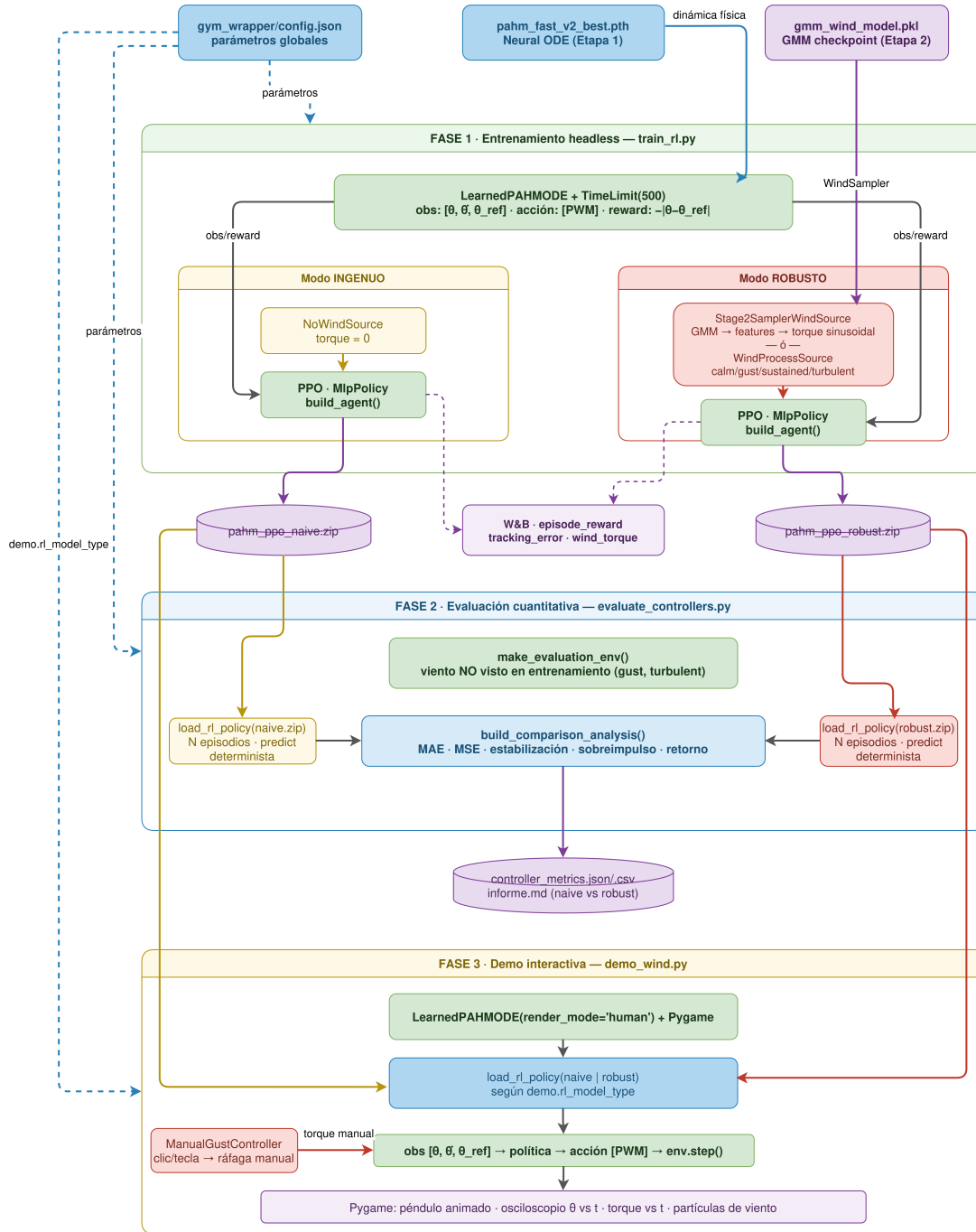


Figura 5: Diagrama del bucle de control robusto en la Etapa 3: Integración del entorno Gymnasium con la física híbrida y el muestreador latente.

**Evaluación cuantitativa — fuente de viento real usada.** El script `evaluate_controllers.py` (`make_evaluation.env`) construye el entorno de evaluación pasando únicamente `wind_pattern` (de `unseen_wind_patterns`: `gust`, `turbulent`) y **no** propaga `wind_source`. Por diseño, esto hace que la evaluación final de ambos controladores corra sobre perturbaciones de `WindProcess`, no del `WindSampler` de Etapa 2 — consistente con la columna `wind_source=wind_process` observada

en `controller_metrics.csv` para todos los episodios de `naive` y `robust`. Es decir: la diferencia entre ambos controladores en la evaluación final proviene de cómo fue entrenado cada uno (sin perturbaciones vs. con perturbaciones activas), no de la fuente de viento usada al evaluarlos.

**Observación y referencia.** El entorno `LearnedPAHMODE` expone `theta_ref` configurable (por `config`, constructor, `set_theta_ref()` o `reset(options={"theta_ref": ...})`) y reporta `theta_ref`, `tracking_error` y `abs_tracking_error` en `info`. La recompensa de seguimiento combina error cuadrático, velocidad angular y esfuerzo de acción con pesos configurables (`reward.tracking_error_weight`, `reward.velocity_weight`, `reward.control_weight`).

## 7. Comparación entre política *naive* y política robusta

Para evaluar el efecto del entrenamiento robusto, se comparó la política *naive*, entrenada sin perturbaciones de viento, contra la política robusta, entrenada bajo perturbaciones. Ambas políticas fueron evaluadas bajo las mismas condiciones de prueba con viento, utilizando las métricas de retorno acumulado, error de seguimiento, sobreimpulso y tiempo de asentamiento.

Cuadro 6: Comparación cuantitativa entre las políticas *naive* y robusta.

| Métrica                | Naive     | Robusta   | Diferencia |
|------------------------|-----------|-----------|------------|
| Retorno promedio       | -60,10983 | -28,89655 | +31,21328  |
| MAE seguimiento        | 0,02118   | 0,02253   | +0,00135   |
| MSE seguimiento        | 0,00443   | 0,00350   | -0,00093   |
| Sobreimpulso máximo    | 0,40340   | 0,33264   | -0,07076   |
| Tiempo de asentamiento | 1,16889   | 0,77000   | -0,39889   |

Los resultados muestran que la política robusta supera a la política *naive* en 4 de las 5 métricas evaluadas. En particular, la política robusta obtiene un retorno promedio considerablemente mayor, pasando de -60,10983 a -28,89655. Esto representa una mejora de 31,21328 unidades en recompensa acumulada, lo cual indica un mejor desempeño global bajo perturbaciones de viento.

Además, la política robusta reduce el error cuadrático medio de seguimiento de 0,00443 a 0,00350, disminuye el sobreimpulso máximo de 0,40340 a 0,33264, y reduce el tiempo promedio de asentamiento de 1,16889 s a 0,77000 s. Estos resultados sugieren que la política robusta no solo responde mejor ante perturbaciones, sino que también estabiliza el sistema más rápido y con menor oscilación.

La única métrica en la que la política *naive* obtiene un valor ligeramente mejor es el error absoluto medio de seguimiento, con 0,02118 frente a 0,02253 para la política robusta. Sin embargo, esta diferencia es pequeña en comparación con las mejoras obtenidas por la política robusta en retorno acumulado, error cuadrático, sobreimpulso y tiempo de asentamiento.

También se observa una diferencia importante en la variabilidad del retorno: la política *naive* presenta una desviación estándar de 122,32229, mientras que la política robusta presenta una desviación estándar de apenas 11,23731. Esto indica que la política robusta tiene un comportamiento mucho más consistente entre episodios, mientras que la política *naive* es más sensible a las perturbaciones del entorno.

En conclusión, aunque la política *naive* mantiene un error absoluto medio ligeramente menor, la política robusta presenta un mejor desempeño global bajo condiciones con viento. La política robusta supera a *naive* en 4 de 5 métricas y muestra una respuesta más estable, con menor sobreimpulso, menor error cuadrático, menor tiempo de asentamiento y mayor recompensa acumulada.

**Nota sobre la fuente de viento en esta evaluación.** Como se señaló en la sección anterior, los controladores fueron evaluados con perturbaciones de viento activas (`wind_enabled=True`), siguiendo el procedimiento de `evaluate_controllers.py` sobre escenarios no vistos en entrenamiento.

## 8. Weights & Biases

Todas las corridas del proyecto (Etapas 0, 1, 2 y 3) se registran bajo un único workspace de equipo en Weights & Biases:

**Workspace:** `wandb.ai/aprendi1s26`

Dentro de ese workspace existen tres proyectos, uno por etapa grupal del proyecto (la Etapa 0, individual por integrante, también es accesible desde el mismo workspace, cada integrante registró sus corridas de DQN ahí):

| Proyecto W&B                      | Etapas correspondientes                                    |
|-----------------------------------|------------------------------------------------------------|
| <code>etapa-1</code>              | Etapas 1 — Estimador de Perturbaciones de Viento (Grupo 1) |
| <code>etapa-2-unsupervised</code> | Etapas 2 — Representación No Supervisada (Grupo 2)         |
| <code>pahm-rl-stage3</code>       | Etapas 3 — Control Robusto RL (Grupo 2)                    |